

Design and Implementation of a Blockchain-Based Loan Ledger with Proof-of-Work and Digital Signatures

CISC6880 — Blockchain Technology

Instructor: Mohamed Rahouti, Ph.D.

Github Repository

Rayed Jawad
Cybersecurity
Fordham University
New York, NY, USA
rjawad1@fordham.edu

Shamia Shanaha
Computer Science
Fordham University
New York, NY, USA
sshanaha@fordham.edu

Abstract—Cryptocurrency and blockchain technology have been a major topic in financial markets and online discussions over the past few years. Despite this, many people still find blockchain difficult to understand because the concepts can be hard to picture without a hands-on example. This paper describes the design and implementation of a simplified blockchain system that is built to record loan transactions, which was created as a teaching tool to make these concepts visible and testable. The system simulates a decentralized ledger by storing each transaction block as a JSON file on a disk. The blockchain features are cryptographic hashing with SHA-256, which detects whether data has been changed since the hash was generated [2], Proof of work consensus where the average work required is an exponential in the number of leading zeroes required [1], and digital signature authentication using SECP256K1 elliptic curve to give data integrity and confirm that transactions are genuine [3]. While the loan transactions are simulated, the cryptography and hashing are real. A Flask-based web interface lets users submit transactions, generate their key pairs, and browse the blockchain. There is also an integrity checker that verifies the chain by checking hashes, mining difficulty, and signatures. An automated test checks the three main algorithms separately. The results showed how mining time grows as difficulty increases, which is how Proof of Work was made to

work. The results also confirmed that the system can catch any tampering and that once a block is added to the chain, it cannot be changed without redoing all the work. Textbooks and lecture slides can describe what hashing does without showing how hash-linked files usually create tamper evidence, which can make it hard for people to understand how it works. This project closes the gap by giving a working blockchain.

Index Terms—Blockchain, Proof of Work, Digital Signatures, SHA-256, ECDSA, Tamper Detection

I. INTRODUCTION

It's almost every day we hear of cryptocurrency and talk of the blockchain everywhere around the internet and within lucrative financial markets. While cryptocurrency and the blockchain have been a hot-button topic for the last 10+ years, ever since the launch of Bitcoin, not everybody is on board with the crypto boom. This is mainly because, to everyday people, this proposition of the blockchain can be difficult to grasp or visualize mentally, and is hard to understand for anyone who isn't already tech savvy in the fintech field. Our blockchain loan borrower project/demonstration can help clear up any misconceptions and fill in any cognitive blind spots. As blockchain grows and eventually

merges into our everyday finances, the importance of understanding and being educated in its concepts is now becoming more important than ever.

A. Background

Blockchain technology was first introduced by the pseudonym Satoshi Nakamoto in the 2008 white paper “Bitcoin: A Peer-to-Peer Electronic Cash System” [1]. The white paper itself described a proposition for a certain kind of system in which financial transactions could be recorded publicly in a distributed ledger without the need for a trusted central authority like a bank. The main idea of the “blockchain” was that we were to record every transaction in a chain that included the cryptographic hash of the record before it. So if adding a new record requires computational work, then modifying any previous record would require the computer to redo all the work that followed before it. This makes any kind of tampering with the data computationally impossible. This was the cryptographic linking and proof of work consensus that made cryptocurrency what it is today and became the foundation of Bitcoin and other future blockchain-based systems. In this project, we implemented a blockchain-based loan ledger as a teaching demonstration on how a blockchain-based system works. We made it so that users can emulate submitting loan transactions, such as borrower, lender, and amount, through their web browser. While these demo “transactions” are simulated, the backend cryptography and hashing they compute are real. Each transaction is placed in a block that is linked to the previous block by a SHA-256 hash. This takes any input and produces a fixed-length 256-bit output [2]. Because of this, even a single character change in the data results in a completely different hash, destroying any relationship to the original linked hash. A proof-of-work algorithm is utilized that requires finding a nonce so that the block and its hash have a certain number of leading zeros. We also utilized an Elliptic Curve Digital Signature Algorithm (ECDSA) to have users sign their transactions. With the corresponding public key and signature stored in the block, this allows a holder of a private key to verify that the transaction utilizes the public key. Only that particular private key would be able to produce a specific signature utilizing its corresponding public key.

No one without that private key can produce that same signature. This is why Bitcoin utilizes the secp256k1 elliptic curve [1], [3] and this implementation project follows this same structure due to this. The Flask-based web UI also provided us with tools to generate key pairs, verify signatures, and display the blockchain clearly for visual understanding.

B. Motivation

Initially, the overall goal of this project was to develop a working, end-to-end implementation of a blockchain system from the ground up. A system that is able to exercise and display important cryptographic principles directly locally from our project, without just relying on and connecting to pre-built blockchain networks. But over time, we realized that our main motivation for this project is to educate. We want to be able to demonstrate the core components of a blockchain and assess its behaviors. Most educational treatments of blockchain stop at a conceptual level. From textbooks to lecture slides, the demonstrations of blockchain without an intractable concrete showcase can make it difficult to grasp, which can lead many away and even cause them to avoid blockchain as a result of being overwhelmed. For example, describing what hashing does, without illustrating how hash-linked files work, without showcasing how they can provide evidence of chain tampering, can be hard for many to understand. What this project aims to do is try to make every concept concrete and observable, whether illustrated within the website interface itself or while looking in the backend, and showing how files interact and react to one another and change. The concept of a loan recording domain was chosen due to its simple enough nature to keep the data model small and focused, and not spiral out of control into complexity. It also provides a realistic scenario in which immutability and the identity verification of transactions have clear value.

II. IMPLEMENTATION

A. Overview

The system is built in 3 layers. The first layer is the data layer, which contains a folder of JSON files on disk, one per block, each named with a sequential number. The second is the blockchain engine, which is the Python module called `block.py` that

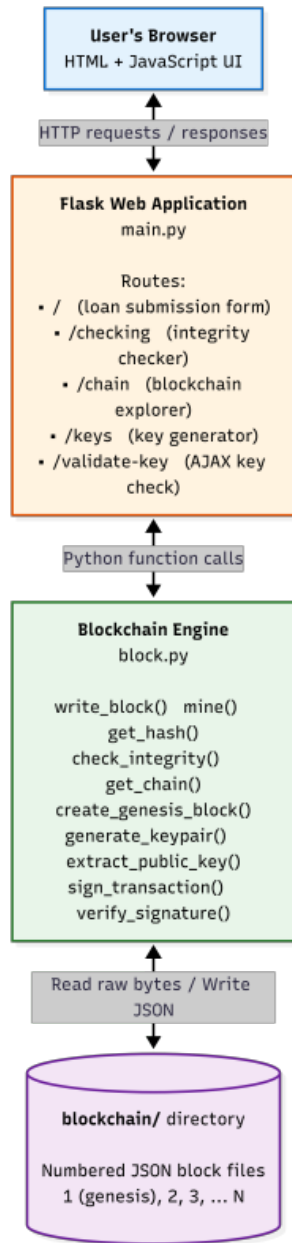


Fig. 1. **System architecture** of the blockchain web application.

contains all the logic for creating, reading, hashing, mining, signing, and checking the blocks. The third is the web application, which uses a Flask server in the main file that connects the engine to the user through the browser interface, along with the HTML templates for the UI. Keeping these layers separate means the blockchain engine does not depend on Flask at all and can be imported and tested on its own. This makes writing automated

tests straightforward. The Flask layer is simple, where it receives input from the user, passes it to the engine, and sends the results back to the browser. The blockchain logic is in the `main.py`.

When a user submits a loan transaction through the browser, Flask receives the form data and calls the `write_block()` function. This function is used to validate the input, build the transaction data, and sign it if a private key is provided, and runs the mining process to find a valid nonce, and writes the finished block as a new JSON file in the blockchain folder. When the user looks at the chain explorer or runs the integrity checker, Flask would read those files back through the `block.py` file and show the results in the browser.

Every block is a JSON object containing the fields borrower, lender, amount, timestamp, which is in Unix time, `prev_block`, which is an object with the filename and a SHA-256 hash of the previous file block, and `proof_of_work`, which is an object that contains the nonce and resulting hash. If the user provides a private key when submitting the transaction, the block will also include a public key field in PEM format and a signature field as a hex string. The genesis block is a special case where the borrower and lender are both set to Genesis, the amount is 0, and the `prev_block` hash is empty since there is no block before it. The `prev_block.hash` field holds the SHA-256 hash of the full raw bytes of the previous block file and not just its field values. This means that even a whitespace can change the file and will be caught by the integrity checker.

The web application has five routes that users can interact with. The main page is where the users fill out and submit their loan transactions. The checking page runs the integrity checker and shows whether each block in the chain is valid or has been tampered with. The chain page shows all the blocks in order with their information and signature status. There is also another check that happens when a user puts their private key into the form, which checks if the key is valid without having to reload the page. Finally, the keys page makes a brand new key pair every time it is visited and never saves the private key on the server.

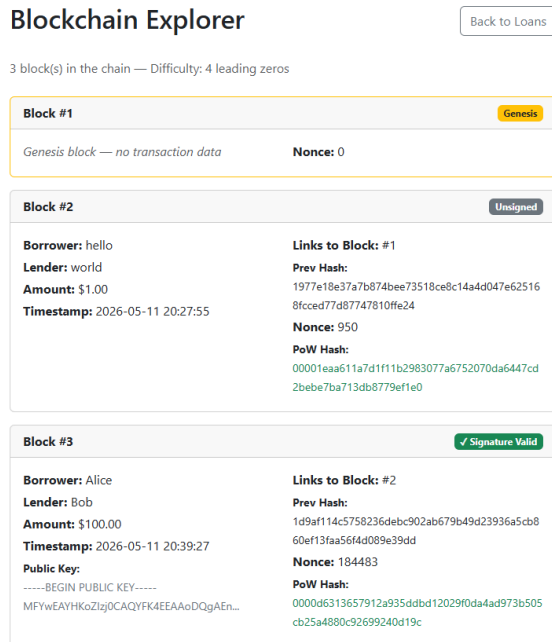


Fig. 2. **Chain explorer page** showing 3 block cards in a row - one Genesis block, one signed block (green badge), one unsigned block (grey badge). Make sure the PoW hashes starting with 0000 ... are visible.

B. Basis of implementation

The system uses three core cryptographic algorithms and one important protocol decision, each of which is directly connected to a feature of the system.

The first SHA-256 hash function [2] is used in two different ways. The first way is to hash each block's raw file and store that hash in the next block as `prev_block.hash`, which is what creates the tamper-proof chain. The second way is inside the mining loop, where the block's data is combined with the nonce and hashed over and over until the results start with the required number of zeros.

The second is Proof of Work [1], [4], [5], where the system tries nonce values one by one until it finds where the SHA-256 hash of the block data combined with that nonce starts with four zeros, which is controlled by the `DIFFICULTY = 4` constant. There is no quicker way to this value than to keep trying, which makes mining computationally expensive. However, when the valid nonce is found, verifying it only needs computing the

hash once to confirm it starts with the required zeroes, which is what makes the overall mechanism practical to use.

The third is the ECDSA and SECP256k1 curve, which is the same curve [3] that Bitcoin uses [1], which lets users sign their transactions. When the user uses the private key, the system takes the four transaction fields, which are the borrower, lender amount, and timestamp, and turns them into JSON byte strings. That byte string is then signed using the user's private key through the ECDSA Python library, which gives a unique signature that gets stored in the block alongside the public key. The important thing about this signature is that it is tied to both the private key and the information of the four fields. So when someone wants to verify the transactions later, the system reconstructs that same byte string from the stored fields and checks it against the stored signature and public key. If one character in the four fields is changed after signing, the signature checker will fail, and the block will be flagged as invalid.

The design decision that ties all of this together is the use of `JSON.dumps(sort_keys = True)`. JSON does not guarantee that the order in which the keys appear, which means that the same block of data could be written out differently each time, even if the actual value has not been changed. For example, one time the JSON might write the borrower before the amount, and another time it might write the amount before the borrower. Even though the data is the same, the two strings are different at the byte level, which causes completely different hashes and breaks the Proof of Work check and the signature check. By always sorting the keys using `sort_keys = True` in the `mine()`, `check_integrity()`, `sign_transaction()`, and `verify_signature()` functions, the system guarantees that the same block will always have the same byte string, no matter when or where it is run, which keeps everything consistent.

C. Description of the implementation

The system is written in Python and organized into two main files, which are `block.py` and `main.py`, along with the `blockchain` folder where the block files are saved, and a `template` folder that holds the HTML pages styled.

In `block.py`, there are ten functions that handle everything related to blockchain. First is the `create_genesis_block()`, which checks if the first block file exists in the blockchain folder and creates one if it does not. The `get_hash(filename)` opens a block file and returns its SHA-256 hash, which is used to link the blocks together. The `mine(data, difficulty)` is the mining function that keeps trying different nonce values until it finds one that makes the hash start with the required number of zeroes. The `generate_keypair()` makes a brand new private key and public key pair and returns them both as PEM strings that the user can save and use. The `extract_public_key(private_key_pem)` takes a private key and gets the public key from it. The `sign_transaction(private_key_pem, tx_data)` takes the transaction data, converts it into a JSON byte string, and uses the private key to get a signature that gets stored in the block. The `verify_signature(public_key_pem, txt_data, signature_hex)` is used to check whether a signed transaction has been tampered with. It takes the transaction fields that are stored in the block and converts them into the same JSON byte string that was used when the transaction was originally signed, and then checks whether the stored signature still matches. If the transaction has not been tampered with since it got signed, it returns `True`. If anything in the transaction was changed after it was signed, it returns `False`, and the block gets flagged as invalid. The `get_chain()` reads all the block files from the folder in order and returns them as a list so they can be shown. The `check_integrity()` is the function that checks whether the blockchain has been tampered with. It goes through every block one by one and runs three checks on each one first. First, it recomputes the hash of the previous block and checks if it still matches what is stored, which catches if any changes were made to the earlier blocks. Second, it reruns the proof of work to make sure the stored nonce still has a valid hash. Third, if the block was signed, it will verify the signature again to make sure none of the transaction fields were changed after it was signed. Any block that

fails one of these checks gets flagged as an error. The `write_block(borrower, lender, amount, private_key_pem)` is the main function that runs every time a user submits a loan transaction. It first makes sure the borrower or lender names are not empty and that the amount is a positive number. Then it builds the transaction and signs it if the user provided a private key, runs the mining process to find a valid nonce, and saves everything as a new JSON file in the blockchain folder.

The `main.py` connects the blockchain to the web interface. It imports all the functions it needs from the `block.py` file and uses them to respond to what the user inputs into the website. If a user submits bad input, like an empty name or a negative amount, the main file catches it and sends an error message.

The loan form in the `index.html` file has a real-time key checking feature built into it. When the user pastes their private key, the code sends the key to the `validate-key` route and checks if the key is valid or not. If it is valid, it gives a verified badge that appears next to the Borrower field, along with the first 20 characters of the public key, so the user can confirm their key was recognized. If the key is not valid, then an unverified badge appears instead. If the user tries to submit the form without a valid key, a warning message will pop up letting them know their transaction is being saved without a signature.

D. Development Milestones

a) Milestone 1 - Initial blockchain structure.:
With the initial commit we were able establish the `/blockchain` directory structure and the code JSON block format. We were able to implement basic chain reading and writing which were both fully functional at this primitive stage.

b) Milestone 2 - SHA-256 and Proof-of-Work.:
We replaced the previous simplistic hashing approach of MD5 to the current SHA-256 hashing algorithm for a more secure and realistic approach. The `mine()` function was also introduced with a difficulty of four leading zeros. The integrity checker was also extended in order to verify proof of work in addition to the hash linkage.

Borrower ✓ Verified

Name Alice

Key: MFYwEAYHkoZIzj0CAQYF...

Lender

Name

Amount

\$

Borrower's Private Key (optional: signs the transaction) ✓ Key Valid

```

-----BEGIN EC PRIVATE KEY-----
MHQCAQEEIK/Nu0uyBa0lh5p+4zcY0AA1+if
37PLLjDGUnI3tYCNGoAcGBSuBBAKoUQDQg
AEnNvB
DR3NgjpY1V34LuEMXBf2HiSSIAZ2ovxT2H/
C6QBsxYiZDJWSeZKeJ+P71uqHw7M7MHqIvg
wxcjZl
24T4Nw==
-----END EC PRIVATE KEY-----

```

No key? [Generate one here](#). Without a key the transaction is saved but unsigned.

Fig. 3. **Loan submission form** with a private key pasted, showing the green "Verified" badge next to "Borrower" and the truncated public-key fingerprint.

c) *Milestone 3 - Flask web application.*: The front-end of the Flask application with its loan submission form, the integrity checker, and bootstrap-style UI was built by both team members and merged together. The `/checking` route was also added for added organization and file structure. We implemented quick visual fixes as the HTML was standardized with the correct DOCTYPE, Bootstrap CDN link, and non-duplicated element IDs.

d) *Milestone 4 - Timestamps and Chain Explorer.*: We added UNIX timestamps to the transaction fields so that the exact time of each transaction is properly recorded and signed. The `/chain` route and `chain.html` explorer template was added. This let users visually explore all the current blocks in the chain with human-readable timestamps.

Key Generator

[Back to Loans](#)

Important: A new key pair is generated every time you load this page. Copy and save your **Private Key** before leaving. It is never stored on the server. Lose it and you cannot sign transactions as yourself again.

Your Private Key (keep this secret)

```

-----BEGIN EC PRIVATE KEY-----
MHQCAQEEIL6CmmJKo1ff0wayfKLh51A5Ys7TJqpFg8H7dhDU6uxo
AcGBSuBBAKoUQDQgAEIi/
yKNn7aWA++Ae0Xzv4qS72SmCr49VMvY8Npz5neYVN5TvIXL4skkcu
bai5nZgwoWi7/LK7GIAEZCM
sQbyVg==
-----END EC PRIVATE KEY-----

```

[Copy](#) [Download .pem](#)

Paste this into the "Private Key" field on the loan form when submitting a transaction.

Your Public Key (safe to share)

```

-----BEGIN PUBLIC KEY-----
MFYwEAYHkoZIzj0CAQYFK4EEAAoDQgAEIi/yKNn7aWA++Ae0Xzv4
qS72SmCr49VMvY8Npz5neYV
N5TvIXL4skkcu bai5nZgwoWi7/LK7GIAEZCMsQbyVg==
-----END PUBLIC KEY-----

```

[Copy](#)

This is your identity on the blockchain. It is stored inside every block you sign.

[Generate a New Pair](#)

Fig. 4. **Key generator page** showing the freshly generated private key textarea and public key textarea side-by-side with the warning banner at the top.

e) *Milestone 5 - ECDSA Digital Signatures.*: We decided to add a digital signature system that was implemented using the `secp256k1` curve. Along with that, the `/key` key generator page was implemented as was the `/validate-key`. We made sure this was live key validation utilizing a JavaScript widget giving the loan form more ease of use.

f) *Milestone 6 - UI Redesign for Identity Clarity.*: We decided to move the public identity indicator. If went from a standalone text area below the private key to a fingerprint line directly under the borrower input field. This was also paired with a rounded pill "verified" badge which appeared on the borrower field when the correct corresponding key was pasted. This change made it much more visually clear that the green checkmark is attached

to a cryptographic key identity, not the name on the text field.

g) *Milestone 7 - Test Suite and .gitignore.*: Finally the `test_blockchain.py` pytest suite was fully added. This along with the `.gitignore` which excludes Python bytecode, visual environment directories, and `.pem` key files.

III. TESTING AND EVALUATION

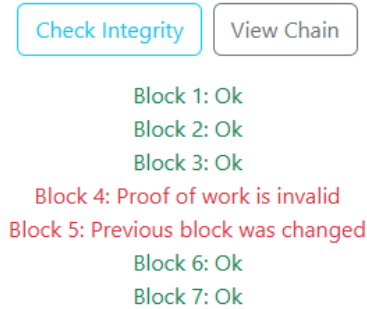


Fig. 5. **Integrity checker results**

showing the list of blocks with blocks 4 and 5 flagged red ("Previous block was changed" / "Proof of work is invalid") and the rest green ("Ok").

To confirm that the system works correctly and catches the tampering as expected, we ran a series of tests and demonstrations covering each important feature of the system. The first thing that was verified was the genesis block. When the system is run for the first time, it automatically creates the genesis block as the starting point of the chain. The integrity checker skips it during checks since it has no previous block to link back to, and every time you run, it confirms that block 1 is always there. Next thing we added several transactions to the chain and ran the integrity checker from the checking page to see if each new block was connecting to the others. Then the next was tampering by going directly into a block file and changing the amount field without updating any of the hashes. When the integrity checker was run after that, it said either "previous block was changed" or "proof of work is invalid" for the block that was changed and any blocks that came after it. In the demo chain, blocks 4 and 5 were intentionally changed this way to serve as an example of this. Both those blocks reported specific errors, while other blocks still showed "Ok." There was also testing done on the signature feature by

generating a key pair, submitting a signed transaction, and then manually changing one of the signed fields in the saved block file. The integrity checker correctly said "Signature is invalid" for that block. We also confirmed that unsigned blocks, meaning the blocks submitted without a private key, are still accepted by the system. When a user submits a transaction without a private key, a warning appears letting them know the transaction will be saved without a signature, but the block is still created and passes the integrity check without any issues. Lastly, there was a test done where mining time was also measured at different difficulty levels to see how the system did as the difficulty increased. The results are shown in Table I below.

TABLE I
MINING TIME VS DIFFICULTY

Difficulty	Time (seconds)
2	0.05
3	0.25
4	1.8
5	9.1

As Table I shows, each extra leading zero that the hash needs to start with increases the mining time. This happens because there is no way to look at a nonce and know ahead of time whether it will produce a valid hash. The only way to find a valid nonce is to keep trying different values one by one until one of them produces a hash that starts with the required number of zeros. The more zeros that are required means the more attempts it takes to find a valid one, which is why the mining time increases so quickly as the difficulty goes up.

A. Unit Test 1 — `test_mine()`

The first unit test, which was `test_mine()`, checks that the `mine()` function works correctly and that it always produces the same results when given the same input. It runs `mine()` with a fixed input and a difficulty of 4 and then makes three checks. The first check confirms that the nonce that came back was a valid non-negative integer. The second check confirms that the hash starts with exactly four zeroes. The third check showed that manually running the hash using the same method gave back the exact same result. The

third check is the most important one because it confirms that the hash produced during mining is the exact same way it will be checked later by `check_integrity()`. If there was any inconsistency in the code, like if `sort_keys` was not used consistently everywhere, then the recomputed hash would come out different from the one that was stored, and the integrity check would flag it as tampered with, even though nothing was actually changed.

B. Unit Test 2 — `test_signature()`

The second unit test was `test_signature()`, which checks that the signing and verification process works correctly and that it can detect when a transaction has been tampered with. It starts by generating a brand new key pair and using the private key to sign a fixed transaction. It then runs `verify_signature()` on the transaction and confirms that it returns `True`, which means that the signature is valid and the data has not been tampered with. After that, it changes the amount field in the transaction to a different number and runs `verify_signature()` again. This time, it confirms that it returns `False`, which means that the signature check failed because the transaction data does not match what was originally signed. This test does not read or write any files on a disk because all of it happens in the memory, which makes running this very quick. It confirms that even changing a single number in the transaction is enough to make the signature check fail.

C. Unit Test 3 — `test_integrity()`

The third unit test is `test_integrity()`, which checks the full chain integrity system in a separate environment from the actual blockchain so that nothing in the real chain is affected. The way it does this is by temporarily pointing the `BLOCKCHAIN_DIR` variable in `block.py` to a temporary folder and then restoring it back to the original when the test is done, even if something goes wrong. The test creates a genesis block and one transaction block in the temporary folder that confirms both say “Ok.” It then changes the amount field in the transaction block file and runs the integrity check again, which the check correctly says “proof of work is invalid” for that block because

the stored hash was computed using the original amount and does not match after the change.

```
===== test session starts =====
platform win32 -- Python 3.9.11, pytest-8.4.2, pluggy-1.6.0
rootdir: ../Blockchain-Technology-Final-Project
collected 3 items

test_blockchain.py::test_mine PASSED [ 33%]
test_blockchain.py::test_signature PASSED [ 66%]
test_blockchain.py::test_integrity PASSED [100%]

===== 3 passed in 0.34s =====
```

Listing 1. pytest output showing all three unit tests passing

The full test finished with everything passing. The integrity checker caught every case of tampering that was tested. The signature verification correctly flagged any block where the transaction data was changed after signing. The web interface correctly showed whether each block was signed, unsigned, or tampered with in every case. The time it took to mine each block took longer as the difficulty increased, which matched what was expected. The invalid inputs worked correctly, such as rejecting empty names and amounts that were zero or native with a clear error message.

IV. REFLECTION

This project was able to successfully implement and demonstrate a simplified blockchain ledger. It clearly demonstrates the core mechanisms of a blockchain system within a local compact Python codebase. Our implementation includes all of the major core concepts that make up a blockchain. This includes: blocks linked by SHA-256 hashes, Proof-of-work mining, and ECDSA digital signatures. We were also able to create a user-friendly web interface through the Flask web framework for peak interactability and ease of use.

The integrity checker was built in a way that ensures that any kind of change to the already hashed and stored data is detected. This highlights the immutable nature of the blockchain. This was powered by the hashing algorithm SHA-256 [2], which is identical to the hashing algorithm that powers Bitcoin’s immutable blockchain [1]. The integrity checker that was implemented correctly identifies both the intentionally tampered blocks and displays them visually within the interface.

Proof-of-Work was also implemented as a commitment mechanism. In the real blockchain, the difficulty-4 proof-of-work requirement makes it so

there needs to be repeated SHA-256 hashing attempts before a valid nonce can be found. This makes block addition computationally expensive. While mining in this simulated educational implementation completes very quickly on modern hardware, the system still accurately models the behavior of real proof-of-work systems. It demonstrates the fundamental blockchain principle that increasing mining difficulty increases computational effort exponentially [4], [5]. Due to each block depending on the previous block's hash, modifying historical data would require you to recompute the proof-of-work for the altered block and all the following blocks afterward. Depending on how far back that block is on the blockchain, this would take a very long amount of time and would be practically impossible to also catch up to the current miners presently mining the most recent blocks in the chain.

ECDSA was also used for transaction authentication. The secp256k1 ECDSA signature system [3] was used to ensure that the signed transaction could only have been produced by the holder of the specific private key whose corresponding public key would be stored in the block. Our second unit test ensured that a single field change in the signed transaction data is sufficient to invalidate the signature as a whole. Public keys were also utilized as the true identities of each signed transaction. The UI makes the very important conceptual distinction between a loan borrower's human-readable string (their name) and their cryptographically confirmed identity (the public key). Their human-readable name is a free-text label where the user can choose to go by anyone without any cryptographic confirmation. Their cryptographic identity, on the other hand, is their public key, which also requires the possession of the corresponding private key. The verified checkmark badge attached to the correct cryptographic key makes it visually explicit that it is not the name but the key that is accurately communicating the identity model that underlies the real blockchain systems included in Bitcoin.

We also conducted three unit tests utilizing a pytest suite in order to validate each core algorithm independently and also to isolate it from our production state. As we saw from the results, the suite completes in under half a second and can also be re-

run safely at any time without affecting the current blockchain.

A. Limitations

While the system successfully showed how the main features of blockchain work, there are a few limitations worth talking about. The first limitation is that the chain is not distributed. There is only one node, and all the blocks are stored on one machine. In a real blockchain like Bitcoin, the chain is spread across many computers, which makes it decentralized and secure, and without that, anyone who has direct access to the machine where the files are stored could go in and change the block files themselves, since there is no other computer on the network that would notice.

The second limitation is that the public key is stored inside every signed block, but in a real blockchain system like Bitcoin, instead of storing the entire key, a much shorter wallet address is stored instead, which is like a compact version of the key. Storing the full key in every block takes up a lot of space.

The third limitation is that the system only stores one transaction per block. In a real blockchain, each block can hold many transactions at once, which makes the system more efficient, and adding multiple transactions per block would require a Merkle tree, which is a way of grouping transactions together and turning them into a single hash that represents them all at once.

The fourth limitation is around identity, since the borrower and lender names are just text that anyone can type in. There is nothing stopping the same person from submitting under a different name or using a different key each time. The cryptography still works correctly, but the system does not have a way to confirm that a name actually belongs to the person submitting the transaction. In a real application, this would need to be handled through some kind of identity verification process outside the blockchain.

B. Future work

While our project achieved many core concepts in terms of blockchain, there are many more advanced features and concepts that would be natural extensions to this project. These additions would make the implementation closer and closer to a

production-quality system and provide users with further valuable learning opportunities.

With extended time, we could eventually implement a distributed network that would implement a peer-to-peer layer so that multiple nodes would be able to share and validate our own personal blockchain. Rather than a single local directory, with this minimal P2P protocol, we could even reach a multi-node consensus. For instance, two Flask instances on localhost exchanging blocks via HTTP would help introduce advanced blockchain interactions where users would be able to feel the challenges of agreeing on certain things in a blockchain where multiple nodes have differing views. The longest chain rule [1], as described by Nakamoto, would of course be implemented as a starting point.

We could also add wallets and balances in the future. Adding a simple currency tracking system for balances would add to the ease of use and prevent any double-spending of funds. This would, of course, require maintaining state across multiple blocks, which would be similar to Bitcoin's UTXO model [6].

To further add to the realism of a blockchain, we could also add transaction fees and a mempool. Adding fees as a field to transactions and also a mempool queue of any unconfirmed transactions would help us move towards modeling the economic layer of Bitcoin's block selection process. In a future model, we could simulate miners prioritizing higher-fee transactions. This kind of extension would also motivate and increase interest in the study of market dynamics and miner incentives.

We could also implement different kinds of consensus methods, such as Proof of Stake, along with Proof of Work, to help people simulate and understand the comparison.

In the future of this project, we could implement Merkle Trees. This would group multiple transactions into a single block, utilizing a Merkle tree to summarize them. As real blocks contain many transactions that are summarized by a Merkle root hash, implementing this tree structure would allow a single block to commit to a large set of transactions efficiently instead of one [6]. This would simulate the improved efficiency of a Merkle tree for verifying a larger set of transactions.

REFERENCES

- [1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," Oct. 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [2] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, Aug. 2015. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- [3] D. Johnson, A. Menezes, and S. Vanstone, "The Elliptic Curve Digital Signature Algorithm (ECDSA)," *International Journal of Information Security*, vol. 1, no. 1, pp. 36–63, Aug. 2001.
- [4] C. Dwork and M. Naor, "Pricing via processing or combatting junk mail," in *Advances in Cryptology – CRYPTO 1992*, E. F. Brickell, Ed. Berlin, Germany: Springer-Verlag, 1993, pp. 139–147.
- [5] A. Back, "Hashcash — A denial of service counter-measure," Aug. 2002. [Online]. Available: <http://www.hashcash.org/papers/hashcash.pdf>
- [6] A. M. Antonopoulos, *Mastering Bitcoin: Programming the Open Blockchain*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2017.